



Lista de Exercícios 3

8 de abril de 2022

Engenharia de Computação

Lilian Giselly Pereira Santos

Pedro Henrique de Brito Nascimento

Projeto e Análise de Algoritmos - Rian Gabriel Pinheiro

Capítulo 4 - Algoritmos Gulosos

1. Considere um grafo não-direcionado $G = (V, E)$ com pesos de aresta não-negativos $w_e \geq 0$. Suponha que você computou uma árvore geradora mínima de G e que também computou os caminhos mínimos para todos os nós partindo de um particular nó $s \in V$. Agora suponha que cada peso de aresta seja aumentado em uma unidade: os novos pesos são $w'_e = w_e + 1$.
 - a) Será que a árvore geradora mínima muda? Dê um exemplo para o qual ela muda ou prove que ela não pode mudar.
 - b) Será que os caminhos mínimos mudam? Dê um exemplo para o qual eles mudam ou prove que isso não pode ocorrer.

Solução:

- (a) Não, a árvore geradora mínima permanece a mesma. Por exemplo, se G tem n vértices, então qualquer árvore geradora tem $n - 1$ arestas. Portanto, ao incrementar cada peso de aresta em uma unidade, aumenta-se o custo de cada árvore gerada também por uma unidade. Dessa forma, todas as árvores geradoras que possuíam custo mínimo no grafo original continuam a possuir custo mínimo com essa alteração, visto que a alteração foi igual para todas e, com isso, podemos concluir que a árvore geradora mínima não é alterada.
- (b) Sim, os caminhos mínimos podem mudar. Um exemplo com intuito de demonstração se dá a seguir:

Ao supormos um grafo G , que originalmente possui arestas indo de s até x , de x até y e de y até t com custo 0, e uma aresta de s até t com custo 1, assim como ilustrado na figura 1, podemos notar que o melhor caminho se dá por $s \rightarrow x \rightarrow y \rightarrow t$, totalizando um custo 0.

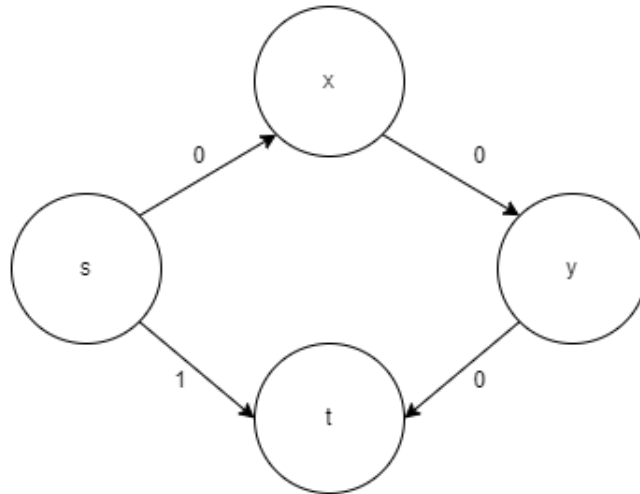


Figura 1. Grafo original.

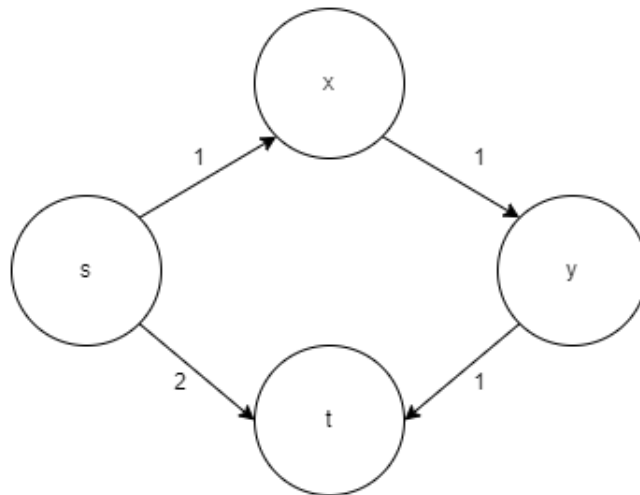


Figura 2. Grafo incrementado.

No entanto, após as alterações feitas no grafo G , como ilustra a figura 2, o caminho $s \rightarrow x \rightarrow y \rightarrow t$ passa a ser mais custoso, com um valor de 3. Por sua vez, o caminho $s \rightarrow t$ que antes possuía desvantagem agora possui custo 2 e é mais benéfico e, portanto, se torna o novo melhor caminho, provando assim que os caminhos mínimos podem mudar.

2. Seja G um grafo não-direcionado. Prove que se todas as arestas são distintas, então ele tem uma única árvore geradora mínima.

Solução: Considerando o enunciado, possuímos um grafo não-direcionado G e desejamos provar que se todas as arestas são distintas, então ele tem uma única árvore geradora mínima. Para provar isso, assumiremos que este grafo possui no mínimo duas árvores geradoras, T_0 e T_1 , e, em seguida, provaremos que pelo menos um par de arestas em G possui peso igual entre si, efetuando, então, a prova por contradição.

Prosseguindo, tanto T_0 quanto T_1 deverão conter uma aresta que a outra não possui, agregando diferença entre si. Sendo e_0 a aresta de peso mínimo em T_0/T_1 e e_1 a aresta de peso mínimo em T_1/T_0 , supomos que o peso de e_0 é menor ou igual ao peso de e_1 . Dessa forma, teremos um novo sub-grafo $T_1 \cup e_0$ que possui um ciclo e uma aresta e_2 que não está em T_0 . Como $e_0 \in T_0$, temos que $e_2 \neq e_0$ e portanto $e_2 \in T_1/T_0$, sendo assim, $w(e_2) > w(e_1) > w(e_0)$. Agora consideramos a árvore geradora $T_2 = T_1 + e - e_2$, onde $w(T_2) = w(T_1) + w(e_0) - w(e_2) \leq w(T_1)$. Mas T_1 é uma árvore geradora **mínima**, então $w(T_1) = w(T_2)$, resultando que T_2 também é uma árvore geradora mínima. Com isso, concluímos por contradição que se todas as arestas são distintas, então ele tem uma única árvore geradora mínima.

3. Considere o problema de agendamento de intervalos. Nós temos um conjunto de atividades $1, 2, \dots, n$; cada atividade i possui um intervalo de tempo a partir de s_i e termina em f_i . Um agendamento — conjunto de atividades — é dito compatível, se nenhuma atividade se sobrepõem no tempo. O objetivo é determinar um agendamento compatível com o maior número possível de atividades. Projete um algoritmo para este problema.

Solução:

```
def endEventTime(task):
    return task.end

def scheduler(tasks):
    tasks.sort(key = endEventTime) # Ordena a lista pelo tempo de término das atividades
    schedule = []
    prevEndTime = -inf #Inicializa o tempo de término para o algoritmo guloso

    for i in range(1, len(tasks)):
        if tasks[i].begin > prevEndTime: # Encaixa as atividades de acordo com a disponibilidade,
            schedule.append(tasks[i])    # ou seja, como as atividades estão ordenadas pelo tempo
            prevEndTime = tasks[i].end    # de término, a preferência será pelas que começam logo
                                         # após o término da menor atividade até o momento

    return schedule

for i in scheduler(tasks):
    print(i)
```

4. Este é um problema que ocorre em análise automática de programas. Para um conjunto de variáveis $x_1, \dots, x_n$, são dadas algumas restrições de igualdade, da forma “ $x_i = x_j$ ” e algumas restrições de desigualdade, da forma $x_i \neq x_j$. Será que é possível satisfazer todas elas? Por exemplo, as restrições

$$x_1 = x_2, x_2 = x_3, x_3 = x_4, x_1 \neq x_4$$

não podem ser satisfeitas. Forneça um algoritmo eficiente que tome como entrada m restrições sobre n variáveis e decida se as restrições podem ser satisfeitas.

Solução:

```
def main():
    m = int(input())
    n = int(input())
    condicoes = []
    for i in range(m):
        condicoes.append(input())

    igualdades, desigualdades = ler_condicoes(condicoes) # Lê e separa as condições de igualdade e desigualdade O(m)
    subset = []

    for u in range(n):
        subset.append(Subset(u, 0)) # Constrói lista de dependência do Grafo,
                                   # com as componentes conexas O(n)
    for igualdade in igualdades:
        union(subset, igualdade[0], igualdade[1]) # Para cada igualdade, constrói as componentes conexas
                                                # Em formato de estrutura de conjuntos disjuntos (union-find)

    flag = 1
    for desigualdade in desigualdades:
        parent1 = find(subset, desigualdade[0]) # Para cada desigualdade, verifica os pais das componentes conexas
        parent2 = find(subset, desigualdade[1]) # Se todos os vértices na componente conexa representam a mesma igualdade
                                                # e portanto tem o mesmo pai. Então basta achar o pai da componente conexa que é aproximadamente O(1)
        if parent1 == parent2: # Se possuem o mesmo pai, são iguais e, portanto a desigualdade é inválida e consequentemente a lista de condições.
            flag = 0
            print("Não é válido")
            break
    if flag: print("É válido") # Se todas as desigualdades são válidas, então o programa é válido
```

5. Alice quer dar uma festa e está decidindo quem chamar. Ela tem n pessoas as quais escolher, e ela fez uma lista de quais pares dessas pessoas conhecem uma a outra. Ela quer selecionar o maior número de pessoas possível, sujeito a duas restrições: na festa, cada pessoa deve ter pelo menos outras cinco pessoas que ela conhece e outras cinco pessoas que ela não conhece. Forneça um algoritmo eficiente que tome como entrada a lista das n pessoas e a lista de pares de quem conhece quem e calcule a melhor escolha de convidados para a festa. Dê o tempo de execução em termos de n .

Solução:

```

void aliceAlgoitm(int n)
{
    Graph G(n);
    bool convidados[n];
    queue<int> rejeitados;
    int entry, aceitos = 0;

    // Recebe os dados do grafo
    for (int i = 0; i < n; i++)
    {
        cout << "Insira aqui quem a pessoa " << G.getPerson(i) << " conhece. Digite -2 para encerrar.\n";
        do
        {
            cout << "Person #: ";
            cin >> entry;

            // Adicionar relação à matriz
            if (entry > 0 && entry <= n && entry != i + 1)
            {
                if (G.isEdge(i, entry - 1))
                    cout << "Convidado " << i + 1 << " e convidado " << entry << " já possuem conexão." << endl;
                else
                    G.addEdge(i, entry - 1);
            }
            else if (entry == i + 1)
                cout << "Essa entrada não é válida!" << endl;
            else if (entry == -1)
                cout << "Convidado " << i + 1 << " tem " << G.getDegree(i) << " conexões." << endl;
            else
                cout << "Entrada inválida, tente novamente! ";
        } while (entry != -1);
    }

    for (int i = 0; i < n; i++)
    {
        convidados[i] = 1; //convida todo mundo
        connections = G.getDegree(i);
        if (connections < 5 || n - connections < 5) // verifica os requisitos
        {
            rejeitados.push(i); // coloca na lista de rejeitados
            convidados[i] = 0; // tira da lista de convidados
        }
    }

    // Verifica os requisitos
    while (!rejeitados.empty())
    {
        int j = rejeitados.front();
        rejeitados.pop();

        for (int i = 0; i < n; i++)
        {
            G.removeEdge(i, j);
            if ((G.getDegree(i) < 5 || n - connections < 5) && convidados[i] == 1) // Tira da lista de convidados
            {
                rejeitados.push(i);
                convidados[i] = 0;
            }
        }
    }
}

```

```

// Contagem de convidados
cout << endl;
for (int i = 0; i < n; i++)
{
    if (convidados[i] == 1)
    {
        aceitos++;
    }
}

// Resultado do algoritmo
if (aceitos == n)
    cout << "Todo mundo atende aos requisitos impostos por Alice, então todo mundo pode comparecer!\n";
else if (aceitos == 0)
    cout << "Ninguém atende aos requisitos, então não é possível Alice dar uma festa.\n";
else
    cout << "Das " << n << " pessoas, " << aceitos << " podem comparecer a festa.\n";
}

int main()
{
    int n; // numero de pessoas
    cin >> n;
    cout << endl;

    aliceAlgoritmo(n);
    cout << endl;
    return 0;
}

```

O tempo de execução em termos de n é $O(n^2)$. Isso se dá pois no *loop for* em que se recebe e insere as conexões no grafo, ocorre n iterações, a fim de encontrar todas as conexões de todos os possíveis convidados, onde cada um pode ter no máximo $n - 1$ conexões. Além disso, há outro *loop for* que verifica se os requisitos (conhecer mais de 5 pessoas, e não conhecer no mínimo 5 pessoas) estão sendo cumpridos. Em seu pior caso, todos os possíveis convidados foram rejeitados, então para cada um deles haverá n iterações para remover da lista de convidados e de todas as suas conexões. Sendo assim, justifica-se o tempo polinomial $O(n^2)$.

6. Um servidor tem n usuários esperando para serem servidos. O tempo de serviço requerido por usuário é conhecido previamente: é t_i minutos para o usuário i . Portanto se, por exemplo, os usuários são servidos em ordem crescente de i , então o i -ésimo usuário tem de esperar a $\sum_{j=1}^i t_j$ minutos. Queremos minimizar o tempo total de espera

$$T = \sum_{i=1}^n (\text{tempo gasto pelo usuário } i \text{ na espera}).$$

Forneça um algoritmo eficiente para computar a ordem ótima na qual processar os usuários.

Solução:

```
1 def getCPUtime(user):
2     return user.CPUTime
3 def server_sched(users_array):
4     users_array.sort(key=getCPUtime) # Ordena de maneira ascendente em n log(n)
5                                     # pelo tempo de serviço. Tempos de serviço menor,
6                                     # reduzem o tempo de espera entre as tarefas
7
8     for user in users_array:        # Para cada usuário O(n) no array de usuários
9         server.execute(user.task) # Executa a task fornecida pelo usuário O(1)
```

7. Considere o seguinte problema.

Entrada: Grafo não direcionado $G = (V, E)$; pesos de arestas w_e ; subconjunto de vértices $U \subset V$.

Saída: A Árvore Geradora mais leve na qual os nós de U são folhas (pode haver outras folhas na árvore também)

Solução: Para encontrar a saída desejada, ou seja, a árvore geradora mais leve na qual os nós de U são folhas, precisamos fazer alguns passos, descritos a seguir:

O primeiro é a construção Grafo $G' = (V', E')$, onde

$$V' = V - U \text{ e } E' = \{(u, v) \mid u, v \in V - U \wedge (u, v) \in E\}$$

Após isso, deve-se aplicar o algoritmo de Kruskal a fim de encontrar a árvore geradora mínima de G' , que é T' . Caso T' não exista, então a árvore geradora mais leve é impraticável e não podemos encontrar uma solução. Isso é explicado pois assumindo que há duas árvores na floresta geradora para o grafo G' , deverá haver um nó $u \in U$ que conecte as duas árvores na floresta, mas isso fará com que u deixe de ser uma folha, resultando em uma contradição.

No entanto, caso T' exista, prosseguimos para a construção de um conjunto de arestas E' , onde:

$$E' : \forall (u, v) \in E' \mid u \in U \wedge v \notin U$$

E o pseudo-código é dado a seguir:

```
for each u in U:
    generateSet(u) # filtra as arestas pelo seu peso
for all edges u,v in E # por ordem de peso
    if encontrar(u) != encontrar(v):
        add edge u,v to T'
        union(u,v)
return T'
```

Com isso, encontramos e retornamos T' , que é árvore geradora mais leve na qual os nós de U são folhas.

8. Um conjunto feedback de arestas de um grafo não-direcionado $G = (V, E)$ é um subconjunto de arestas $E' \subseteq E$ que intercepta todos os ciclos do grafo. Assim, remover as arestas E' tornará o grafo acíclico. Forneça um algoritmo eficiente para o seguinte problema:

Entrada: Grafo não-direcionado $G = (V, E)$ com pesos de aresta positivos w_e .

Saída: Um conjunto feedback de arestas definida $E' \subseteq E$ de peso total mínimo $\sum_{e \in E'} w_e$.

Solução:

```
def minimum_weight_feedback_edge_set(V,E):
    for edge in E:
        edge.weight = -edge.weight

    minSpanTree = kruskal(V,E) # Kruskal vai utilizar as arestas de menor peso que, por serem
                                # previamente negadas, construirá uma MST cujos pesos das arestas
                                # são os maiores pesos reais
    solution = E               # Conjunto solução inicializado com todas as arestas

    for edge in minSpanTree:
        solution.remove(edge) # Remove da solução as arestas contidas na árvore de expansão mínima

    return solution # Retorna o conjunto feedback de peso mínimo (complemento da MST, cujos pesos
                    # das arestas são os máximos)
```

9. Forneça um algoritmo de tempo linear que tome como entrada uma árvore e determine se ela tem um emparelhamento perfeito: um conjunto de arestas que tocam cada vértice exatamente uma vez.

Solução:

```
def perf_match_tree(V,E):
    if len(V)%2==1: return False
    else:
        while len(V):
            if not len(E):
                return False
            leaf = select_leaf(V,E)
            parent = get_parent(leaf,E)

            for edge in get_edges(parent,V,E):
                E.remove(edge)
            V.remove(leaf)
            V.remove(parent)
        return True
```

10. Prove que o problema da mochila pode ser resolvido em tempo polinomial por um algoritmo guloso, se o peso w_i de cada item i for unitário.

Solução: Nesse problema, temos uma mochila com capacidade de aguentar certo peso W e i itens de peso unitário w_i , mas com valores distintos - sendo alguns mais valiosos que outros. Seu objetivo é escolher os itens que resultarão em um maior lucro para o dono na mochila, caso este deseje vender. Uma maneira de resolvê-lo (porém nem sempre tão eficiente) é através de um algoritmo guloso, que é caracterizado por tomar a decisão com a vantagem imediata mais óbvia. Nesse caso, podemos ver então que essa vantagem é a escolha dos itens que possuem maior valor, sem levar em consideração o seu peso, visto que todos possuem o mesmo. Sendo assim, necessitamos inicialmente ordenar de maneira decrescente os itens com base em seu valor e em seguida pegar os primeiros W itens, de forma a encher a mochila até seu máximo e, assim, obter máximo lucro.

```
def greedy_knapsack(values, capacity):
    profit=0
    values.sort(reverse=True) # Ordena os itens pelo seu valor em ordem decrescente O(n.lg(n))
    while capacity and values: # Enquanto houver capacidade e supondo que os itens
        profit += int(values.pop(0)) # são limitados, deposita os itens de maior valor O(n)
        capacity-=1

    print(profit) # Imprime o lucro máximo para o caso onde todas os objetos tem o mesmo peso

values = [int(x) for x in input().split(' ')]
capacity = int(input())

greedy_knapsack(values,capacity)
```

Desse modo, como o algoritmo acima envolve uma ordenação $O(n \log(n))$ e uma verificação $O(n)$ temos que a complexidade final é $O(n \log(n))$. Além disso, como $O(n) < O(n \log(n)) < O(n^2)$, é possível concluir que a complexidade é limitada de modo inferior e superior por duas funções polinomiais (linear e quadrática). Logo,

a complexidade linear-logarítmica é melhor que quadrática e pior que linear (estas sendo funções polinomiais).